

How can we accelerate an algorithm?

- Technology scaling (but Moore's law is flattening out)
- Cache optimizations
- Code optimization
- Exploiting parallelism
- GPUs
- TPUs
- Fancier CPUs? More pipelining? Faster storage? Better networking?
- HW/SW co-design
- Improve the algorithm (trade time for space, approximations, etc.)
- Emerging technology

QUIZ



Traditional technology

Optimization Technique	Potential Speedup	Difficulty	Applicability	Notes
Algorithm Selection	10x-1000x+	Medium-High	High	Changing from $O(n^2)$ to $O(n \log n)$ yields massive gains at scale
Data Structure Optimization	2x-100x	Medium	High	E.g., Hash tables vs. arrays for lookups
GPU Acceleration	10x-1000x	High	Medium	For parallelizable computations only
Multithreading/Parallelization	2x-64x	High	Medium	Limited by number of cores and parallelizable portions
Compiler Optimizations	1.2x-4x	Low	High	Simply changing compiler flags
Memory Hierarchy Optimization	2x-10x	Medium	High	Cache-friendly data structures and access patterns
Memoization/Caching	2x-100x	Low-Medium	Medium	For algorithms with repeated calculations
Hardware Upgrade (CPU)	1.5x-4x	Low	High	Generational improvement in processors
SSD vs HDD Storage	2x-100x	Low	Medium	For I/O-bound algorithms
FPGA/ASIC Implementation	10x-1000x	Very High	Low	Custom hardware, high development cost
Code Micro-optimizations	1.1x-2x	Medium	High	Loop unrolling, reducing function calls, etc.
Language Change (Interpreted to Compiled)	3x-50x	High	Medium	E.g., Python to C++

QUIZ

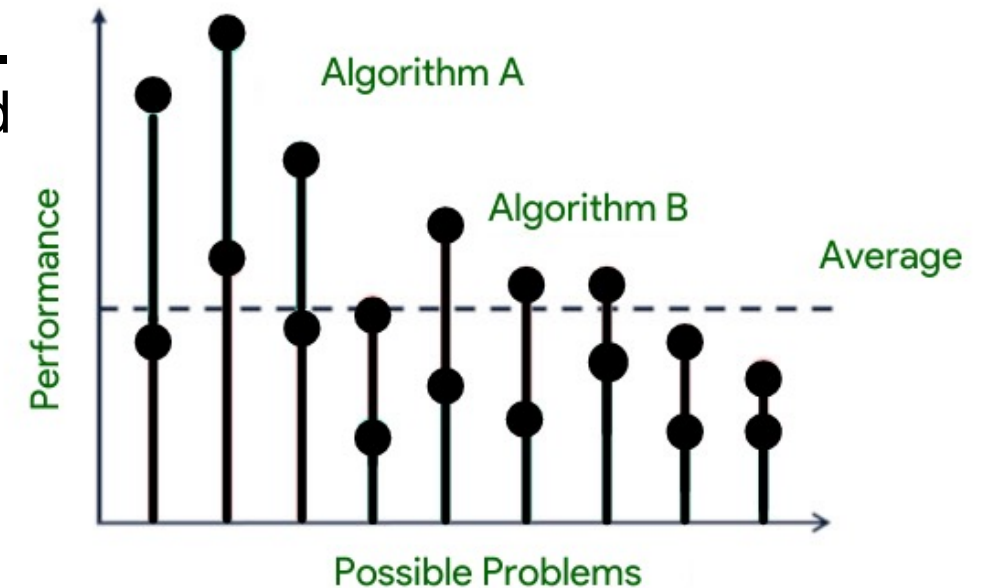
Emerging technology

Technology	Potential Speedup	Energy Efficiency Gain	Technology Readiness	Key Advantages	Major Challenges
Memristors	10x-100x	100x-1000x	Medium-High	In-memory computing, storage/compute unification, analog synaptic behavior	Device variability, endurance, switching uniformity
Neuromorphic Chips	10x-1000x	1000x-10,000x	Medium	Brain-inspired architectures, spike-based computing, massive parallelism	Programming paradigm, algorithms development, scaling to complex tasks
Quantum Computing	Exponential for specific problems	Varies widely	Low	Quantum parallelism, exponential speedup for specialized problems	Decoherence, error correction, limited algorithm scope, cryogenic requirements
Spintronics	5x-50x	10x-100x	Medium	Non-volatile, minimal heat generation, high endurance	Integration with conventional CMOS, scalability issues
Photonic Computing	100x-1000x	100x-1000x	Low-Medium	Ultra-high bandwidth, minimal heat, wave-based parallel processing	Integration challenges, photonic-electronic interfaces
Memcapacitors	10x-100x	50x-500x	Low	Complementary to memristors, charge-based storage, improved switching	Early research stage, fabrication challenges
Reservoir Computing	50x-500x for temporal tasks	100x-1000x	Low-Medium	Efficient time-series processing, reduced training complexity	Limited to specific task domains, generalization issues
DNA Computing	Massive parallelism (theoretical)	Ultra-low (theoretical)	Very Low	Enormous parallelism, molecular-scale computing, energy efficiency	Speed limitations, error rates, interface challenges
Phase-Change Materials	5x-50x	10x-100x	Medium-High	Multi-level storage, established manufacturing, non-volatile	Energy consumption for phase transition, endurance
Protein-based Nanowires	Unknown	10,000x-100,000x (theoretical)	Very Low	Ultra-low power consumption, biocompatibility	Early research stage, manufacturing scalability

No Free Lunch theorem (NFL)

QUIZ

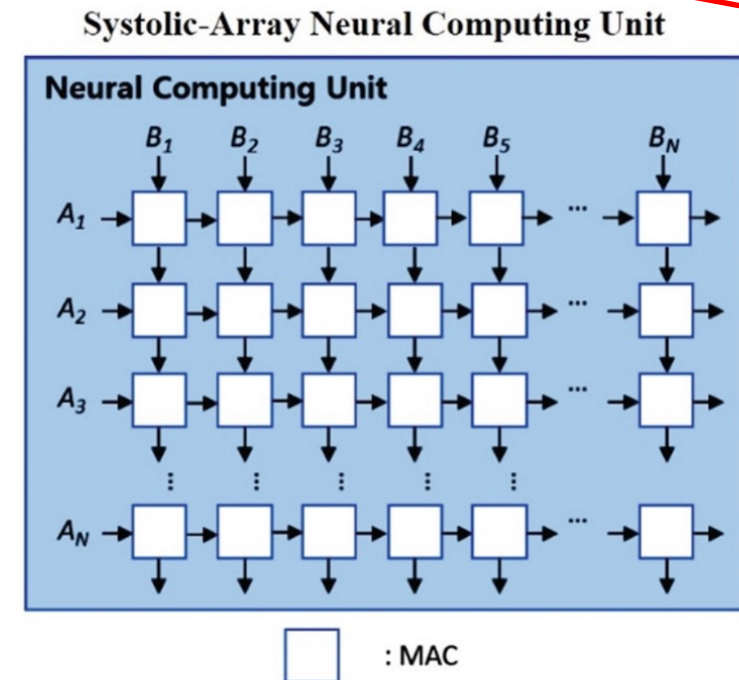
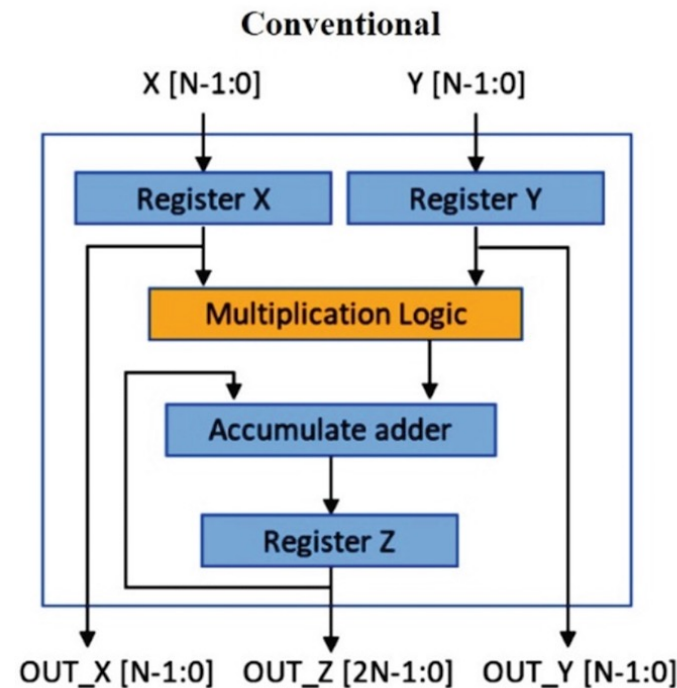
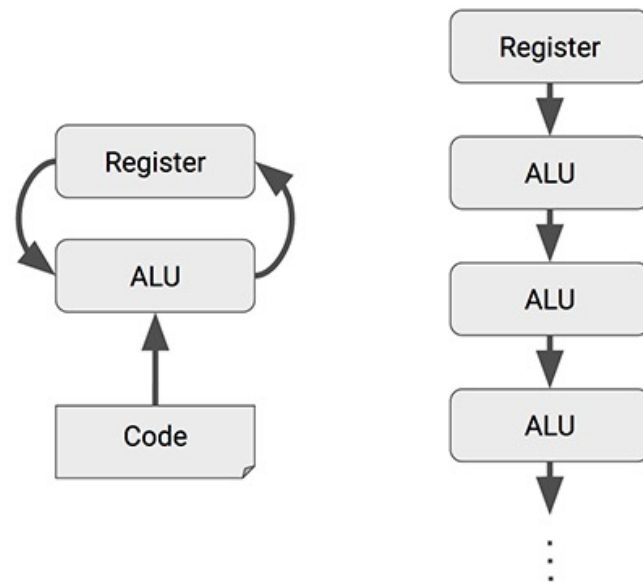
- The "No Free Lunch" (NFL) theorem in optimization and machine learning states that, when averaged across all possible problems, all optimization algorithms perform equally well. I.e., **no algorithm consistently outperforms random search or any other algorithm.**
- The NFL theorem was formalized by David Wolpert and William Macready in 1997.
- **This principle has profound implications for ML and HW optimization:**
 - It explains why specialized algorithms/architectures work well for specific domains but fail in others.
 - It highlights the importance of incorporating domain knowledge when selecting algorithms/architectures.
 - **It reminds us that claims about algorithm/architecture superiority must specify the context/problem class.**



<https://www.geeksforgeeks.org/what-is-no-free-lunch-theorem>

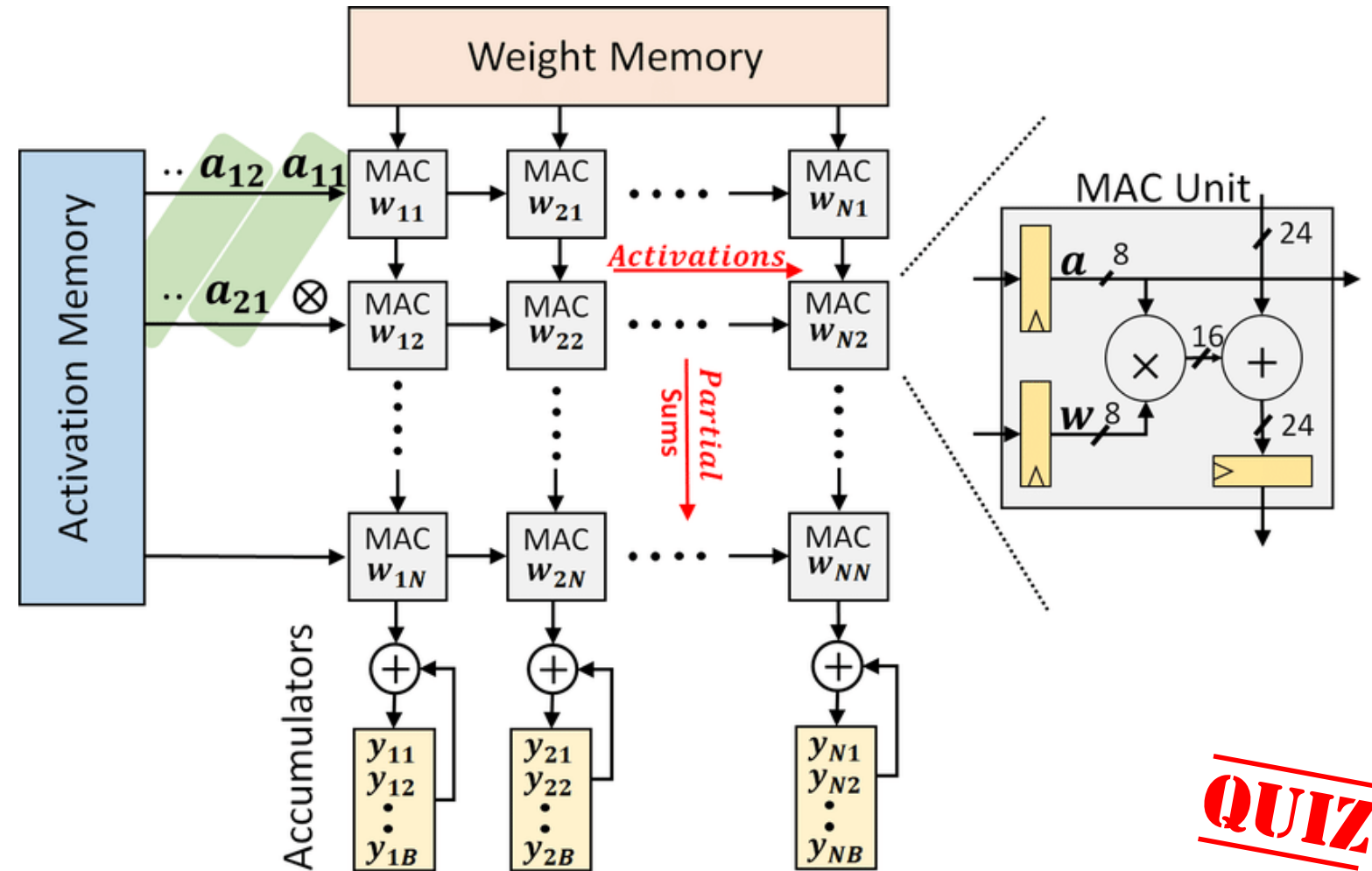
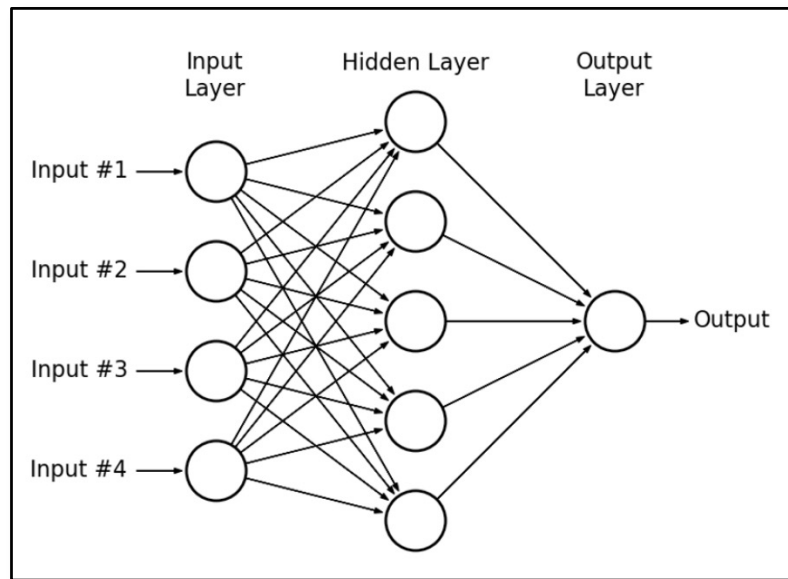
Systolic array

QUIZ



- The name "systolic" comes from an analogy to the human heart, where data pulses through the array of processors in a rhythmic, synchronized fashion.
- CPUs and GPUs often spend energy to access multiple registers per operation. A systolic array chains multiple ALUs together, reusing the result of reading a single register.

How to map a deep neural net on a systolic array



QUIZ

Emerging technology

Technology	Potential Speedup	Energy Efficiency Gain	Technology Readiness	Key Advantages	Major Challenges
Memristors	10x-100x	100x-1000x	Medium-High	In-memory computing, storage/compute unification, analog synaptic behavior	Device variability, endurance, switching uniformity
Neuromorphic Chips	10x-1000x	1000x-10,000x	Medium	Brain-inspired architectures, spike-based computing, massive parallelism	Programming paradigm, algorithms development, scaling to complex tasks
Quantum Computing	Exponential for specific problems	Varies widely	Low	Quantum parallelism, exponential speedup for specialized problems	Decoherence, error correction, limited algorithm scope, cryogenic requirements
Spintronics	5x-50x	10x-100x	Medium	Non-volatile, minimal heat generation, high endurance	Integration with conventional CMOS, scalability issues
Photonic Computing	100x-1000x	100x-1000x	Low-Medium	Ultra-high bandwidth, minimal heat, wave-based parallel processing	Integration challenges, photonic-electronic interfaces
Memcapacitors	10x-100x	50x-500x	Low	Complementary to memristors, charge-based storage, improved switching	Early research stage, fabrication challenges
Reservoir Computing	50x-500x for temporal tasks	100x-1000x	Low-Medium	Efficient time-series processing, reduced training complexity	Limited to specific task domains, generalization issues
DNA Computing	Massive parallelism (theoretical)	Ultra-low (theoretical)	Very Low	Enormous parallelism, molecular-scale computing, energy efficiency	Speed limitations, error rates, interface challenges
Phase-Change Materials	5x-50x	10x-100x	Medium-High	Multi-level storage, established manufacturing, non-volatile	Energy consumption for phase transition, endurance
Protein-based Nanowires	Unknown	10,000x-100,000x (theoretical)	Very Low	Ultra-low power consumption, biocompatibility	Early research stage, manufacturing scalability

QUIZ



Christof Teuscher



teuscher@pdx.edu

www.teuscher-lab.com/teaching



Non-von Neuman architectures

In-memory computation (IMC or PiM)

IMC macro-categories

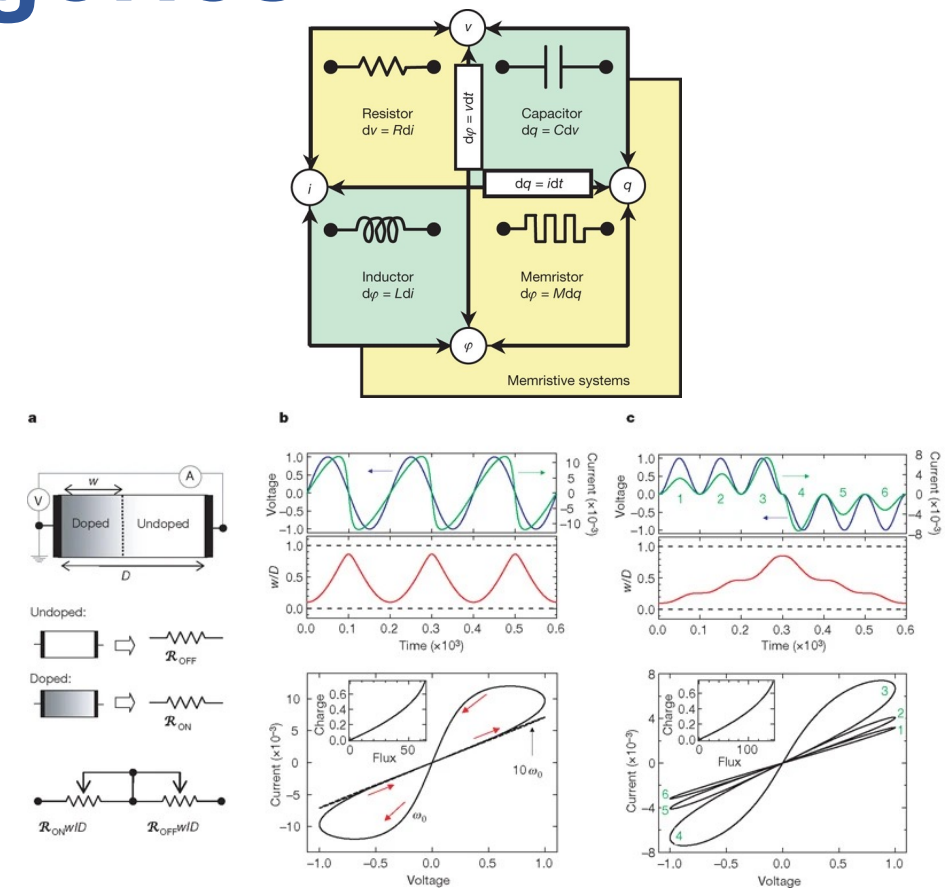
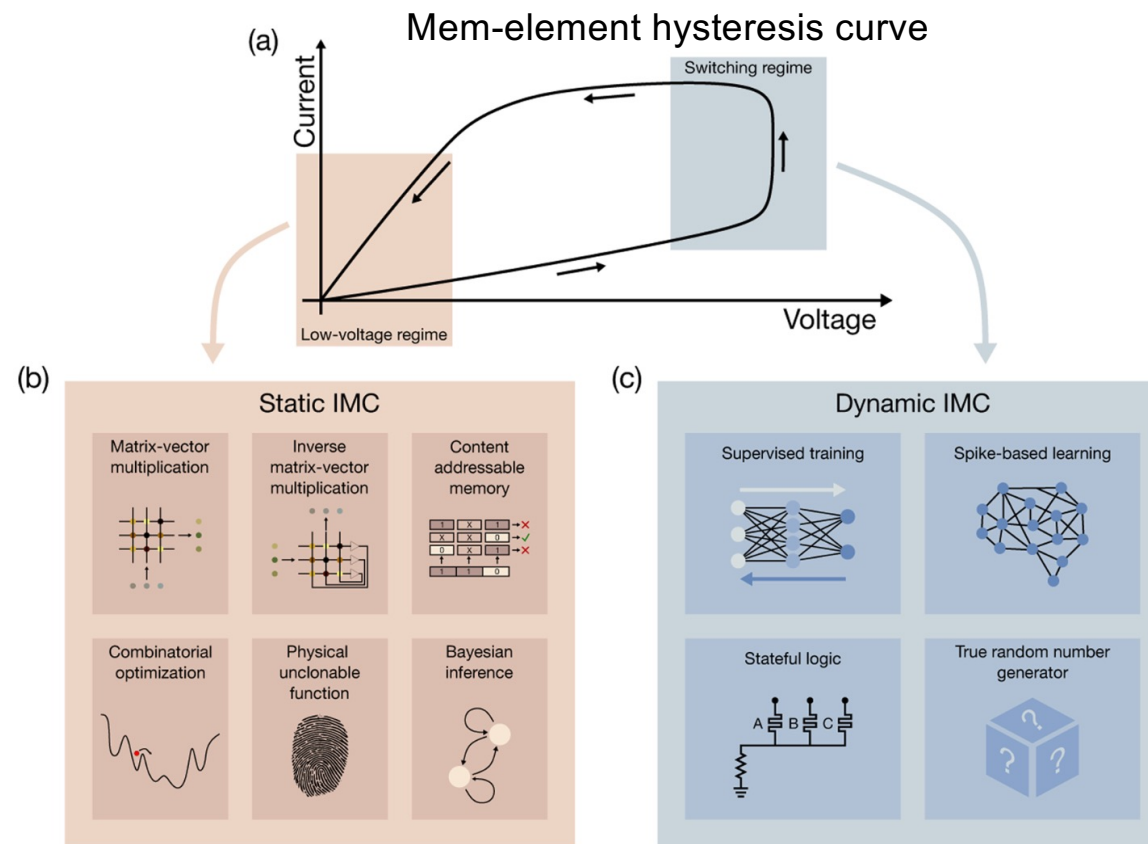


FIG. 4. IMC macro-categories and corresponding applications. (a) Schematic current–voltage (I–V) curve of an emerging memory device, highlighting the low-voltage and high-voltage/switching regimes, corresponding to static and dynamic IMC, respectively. (b) Examples of static IMC, where the memory stores pre-trained data and executes the computation, e.g., MVM. (c) Examples of dynamic IMC, in which the switching regime allows reproducing dynamic features, such as adaptation and learning.

Strukov, D., Snider, G., Stewart, D. *et al.* The missing memristor found. *Nature* **453**, 80–83 (2008). <https://doi.org/10.1038/nature06932>

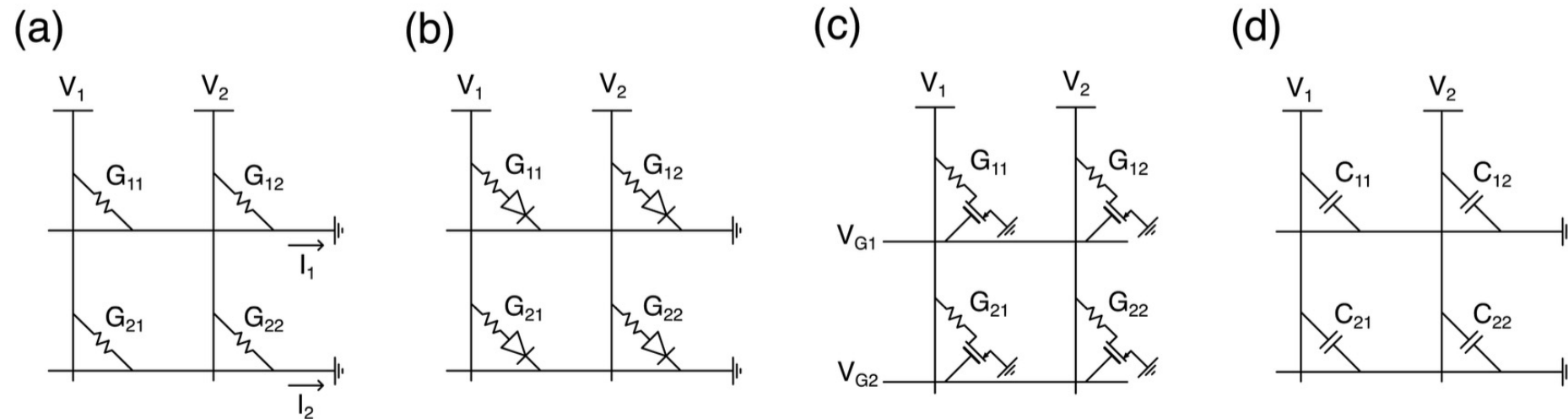
MVM (Matrix-Vector Multiplication) can be executed in a crosspoint memory array by universal circuit laws, such as Kirchhoff's current law for summation and Ohm's law for multiplication.

This is schematically shown in Fig. 5(a), where the application of a voltage V_j at the j th column results in a current at the i th row, connected to ground, given by

$$I_i = \sum_j^N G_{ij} \cdot V_j, \quad (1)$$

where G_{ij} is the conductance of the memory element at position i, j and N is the number of rows and columns.^{7,107} Equation (1) can be written in the compact matrix form $\mathbf{i} = \mathbf{G}\mathbf{v}$, thus evidencing the multiplication of the conductance matrix $\mathbf{G}$ with the voltage vector $\mathbf{v}$.

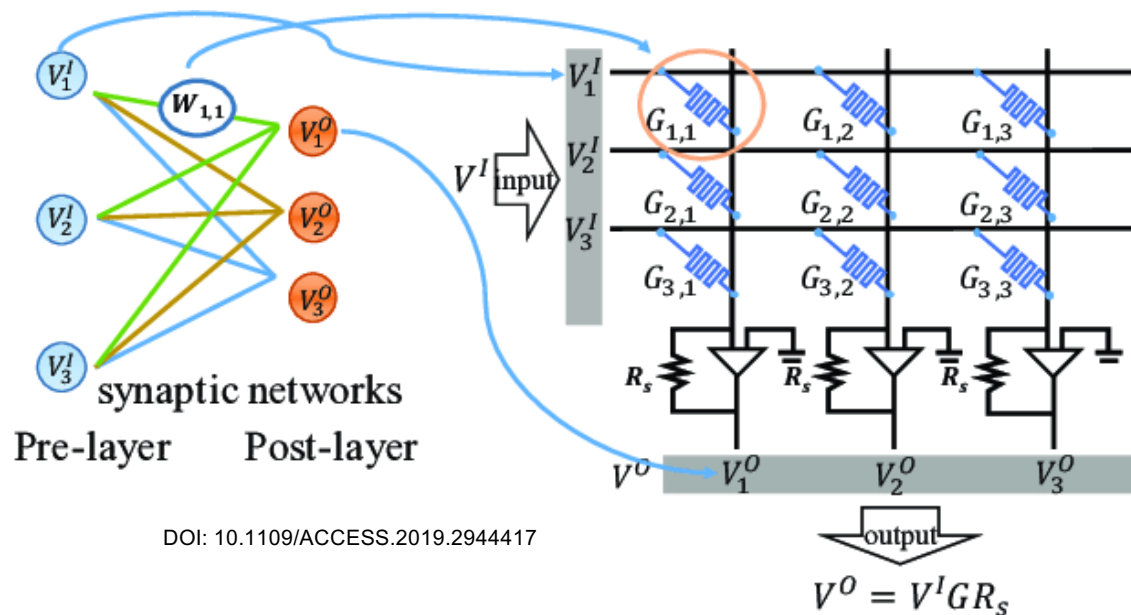
crosspoint
array or
crossbar



QUIZ

FIG. 5. Various cell structures for crosspoint array circuits. (a) One-resistor (1R) structure where the cell consists of a passive resistive device. (b) One-selector/one-resistor (1S1R) structure where the sneak path problem is circumvented by a non-linear selector device without affecting the integration density. (c) One-transistor/one-resistor (1T1R) structure allows for the selection of individual cells during programming and reading at the cost of a lower integration density. (d) One-capacitor (1C) structure, which prevents static leakage during MVM.

How can we map a NN onto a crossbar?

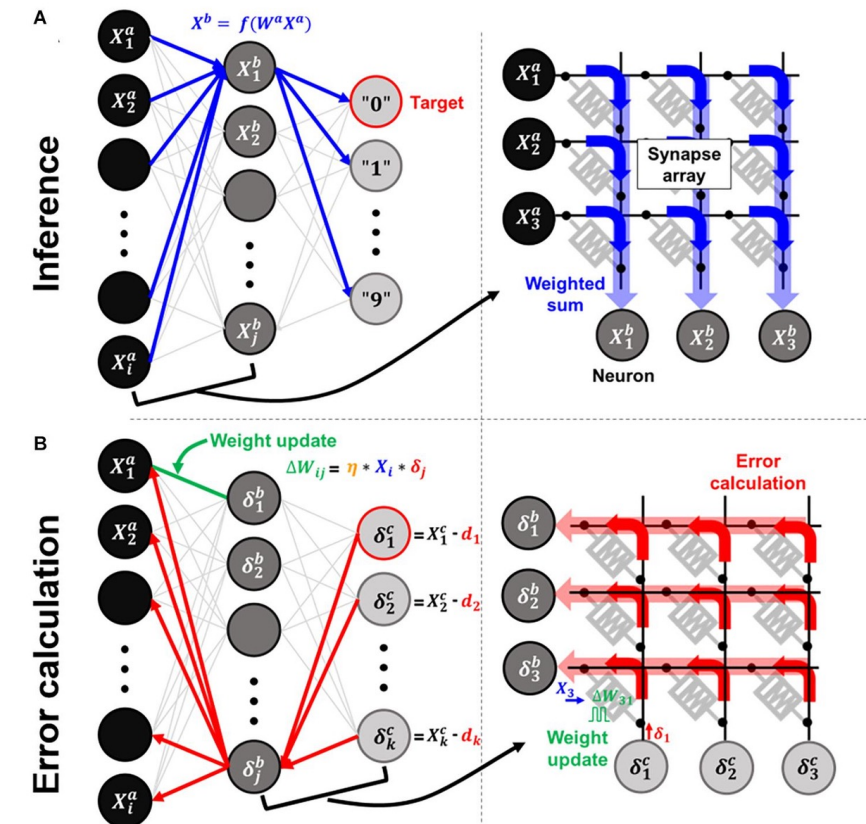


QUIZ

Parallel multiplication: When voltage is applied to the rows, the current flowing through each memristive junction is proportional to the product of the input voltage and the junction's conductance. This effectively performs multiplication at each intersection point simultaneously.

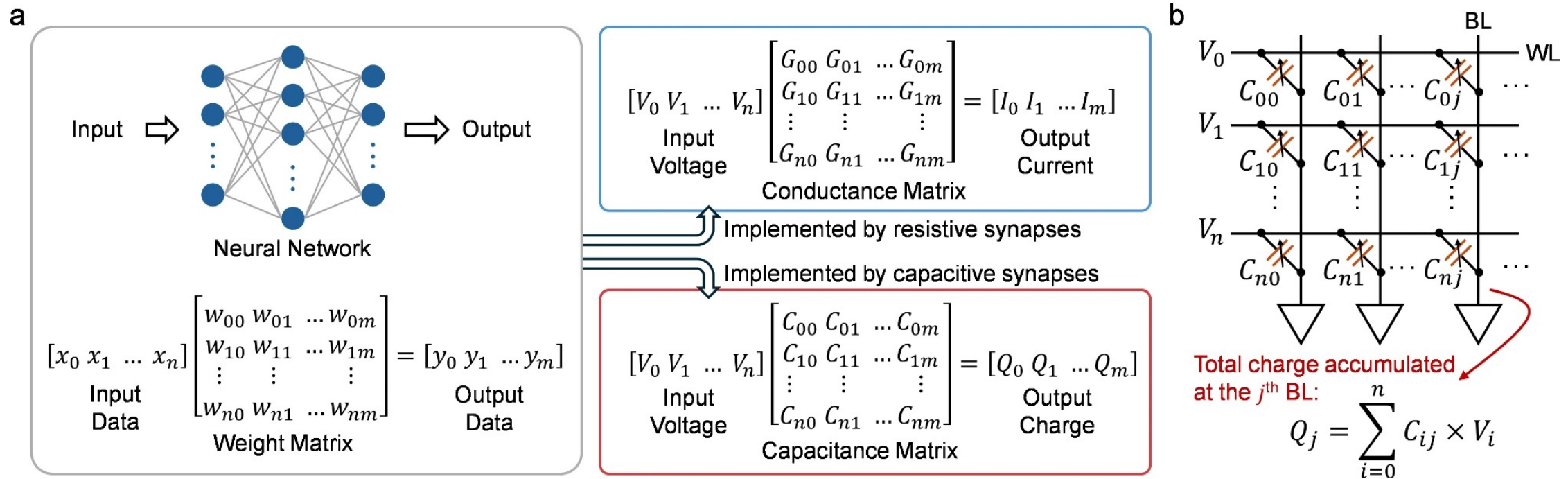
Current summation: The currents from all intersections in a column are naturally summed according to Kirchhoff's current law, effectively adding up all the partial products.

Computational model Hardware neural network



<https://doi.org/10.3389/fnins.2021.690418>

How can we map a NN onto a crossbar?

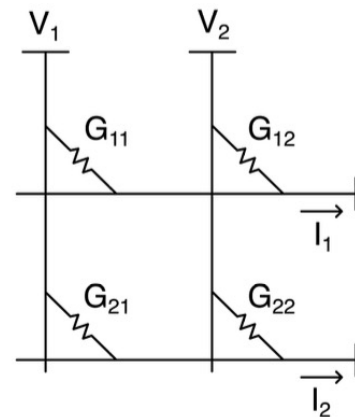


QUIZ

<https://nanoconvergencejournal.springeropen.com/articles/10.1186/s40580-024-00463-0>

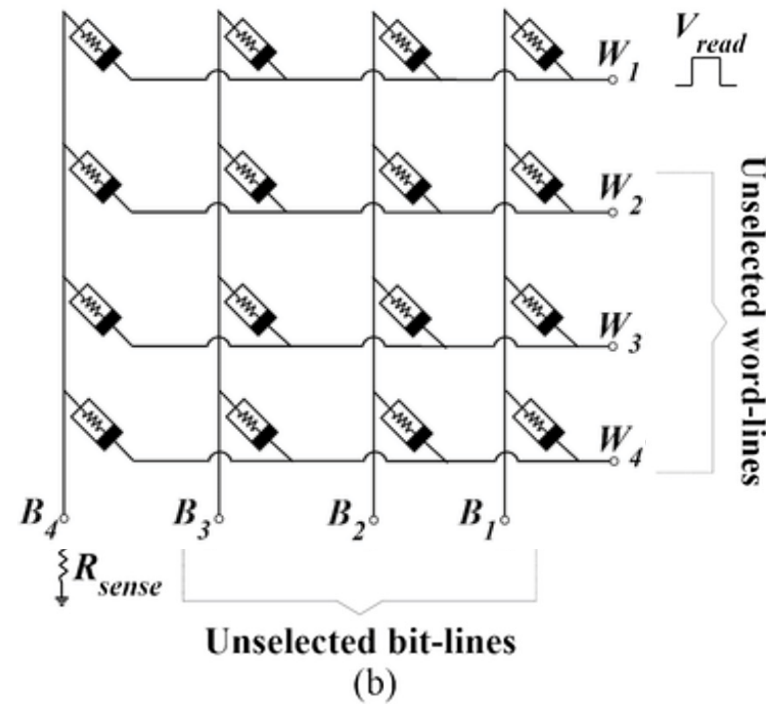
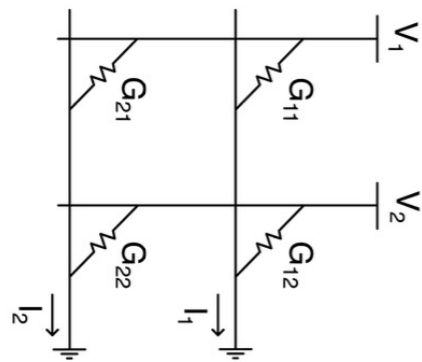


Sneak paths (1)



QUIZ

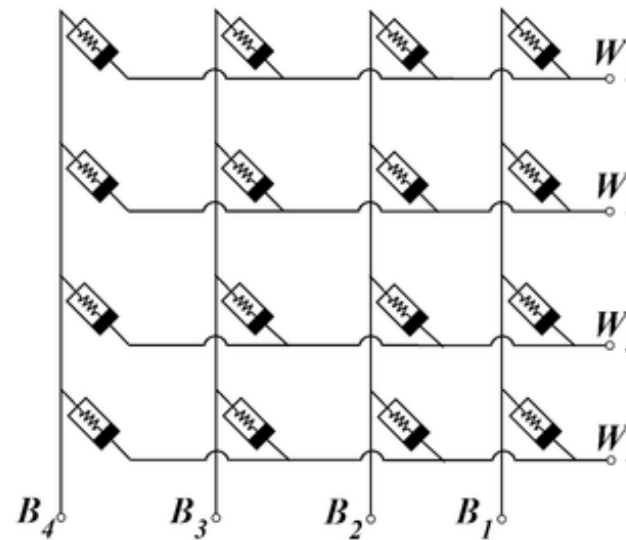
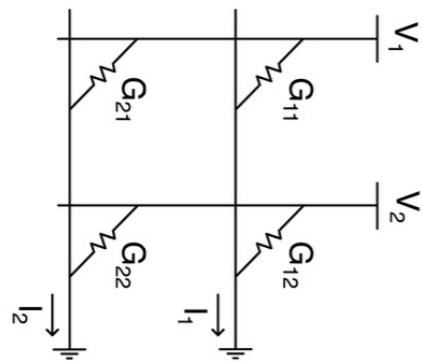
Sneak paths (2)



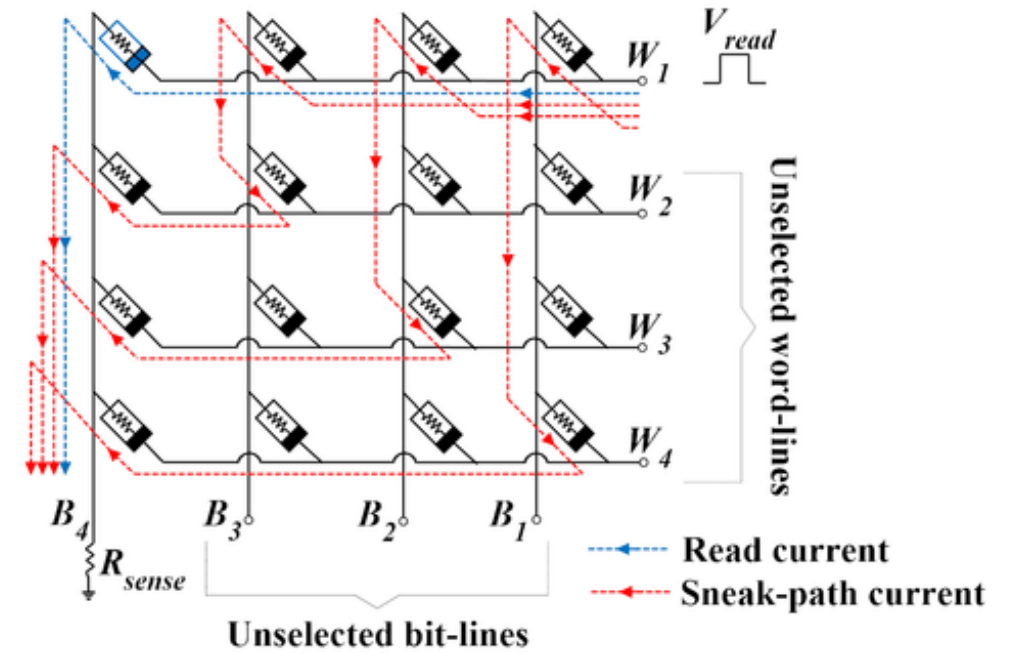
Make a drawing
of where the
current flows

QUIZ

Sneak paths (3)



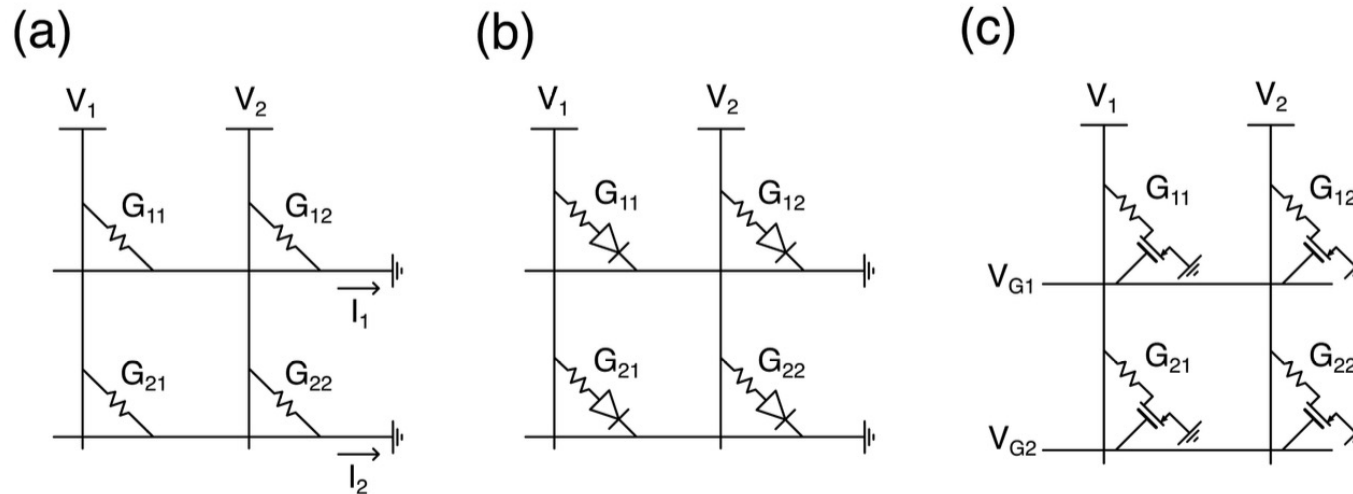
(a)



(b)

QUIZ

Sneak paths (4)



To read or write a cell, you raise only that row's word line V_{GX} , turning on only that row's transistors. Transistors in every other row are off.

Sneak-paths in crossbars are unintended current pathways that occur in memory array architectures.

Solutions: **diodes** or **1T1R structures**

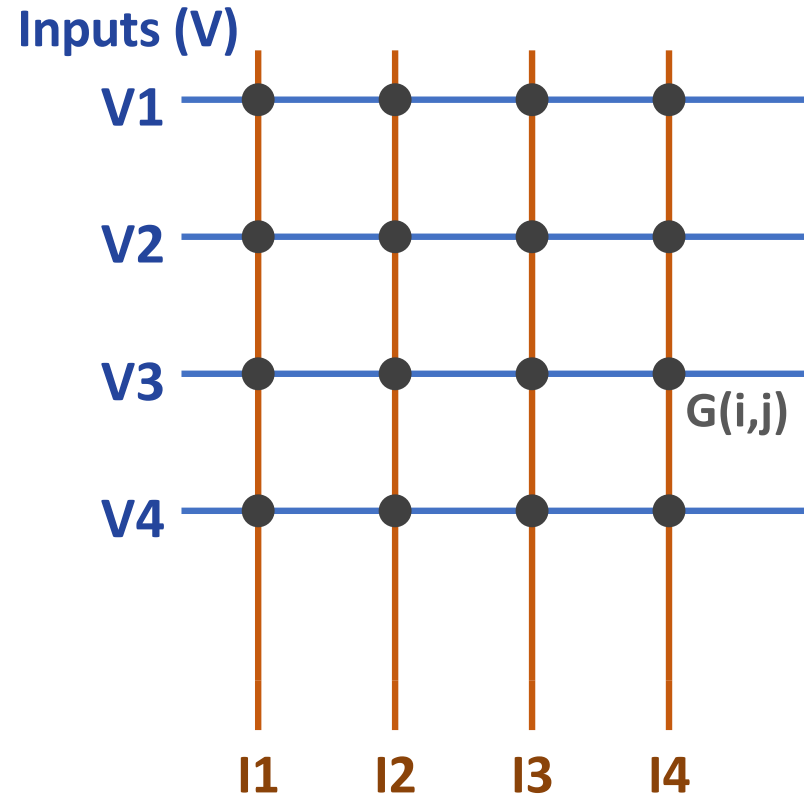
QUIZ

Questions

- What is the primary advantage of in-memory computing.
- In a typical crossbar implementation of a neural network, where are the synaptic weights physically represented?
- What are the benefits of spiking neural nets over analog neural nets?

QUIZ

The crossbar primitive



Outputs: $I = G \cdot V$

QUIZ

Analog MVM in $O(1)$ time

- Program weights as conductances $G(i,j)$ in a resistive memory (ReRAM, PCM)
- Apply the input vector as voltages $V(i)$ on the rows
- Ohm's law: $I(i,j) = G(i,j) \cdot V(i)$
- Kirchhoff's law: $I(j) = \sum_i G(i,j) \cdot V(i)$

→ One column current = one dot product

→ Whole MVM in a single read cycle

Efficient Sparse Mapping on a Crossbar

Before: 8×8 sparse

3						2	
		5					8
4				7			
	6						1

8 non-zeros / 64 cells (~12%)

After: 3×3 packed tile

→
permute

3	2	5
8	4	7
6	1	

8 non-zeros / 9
cells (~89%)

1. Permute & pack (software)

Reorder rows & columns so non-zeros cluster into dense blocks.

Map each block to a small tile. Fewer tiles, less leakage.

8 NZ: 64 → 9 cells (7× smaller)

QUIZ

2. Tile partitioning (software)

Split A into $k \times k$ sub-matrices.

Skip all-zero tiles entirely; schedule only the non-zero ones.

3. Row & bitline gating (hardware)

Power-gate rows whose input is zero and columns with no non-zeros.

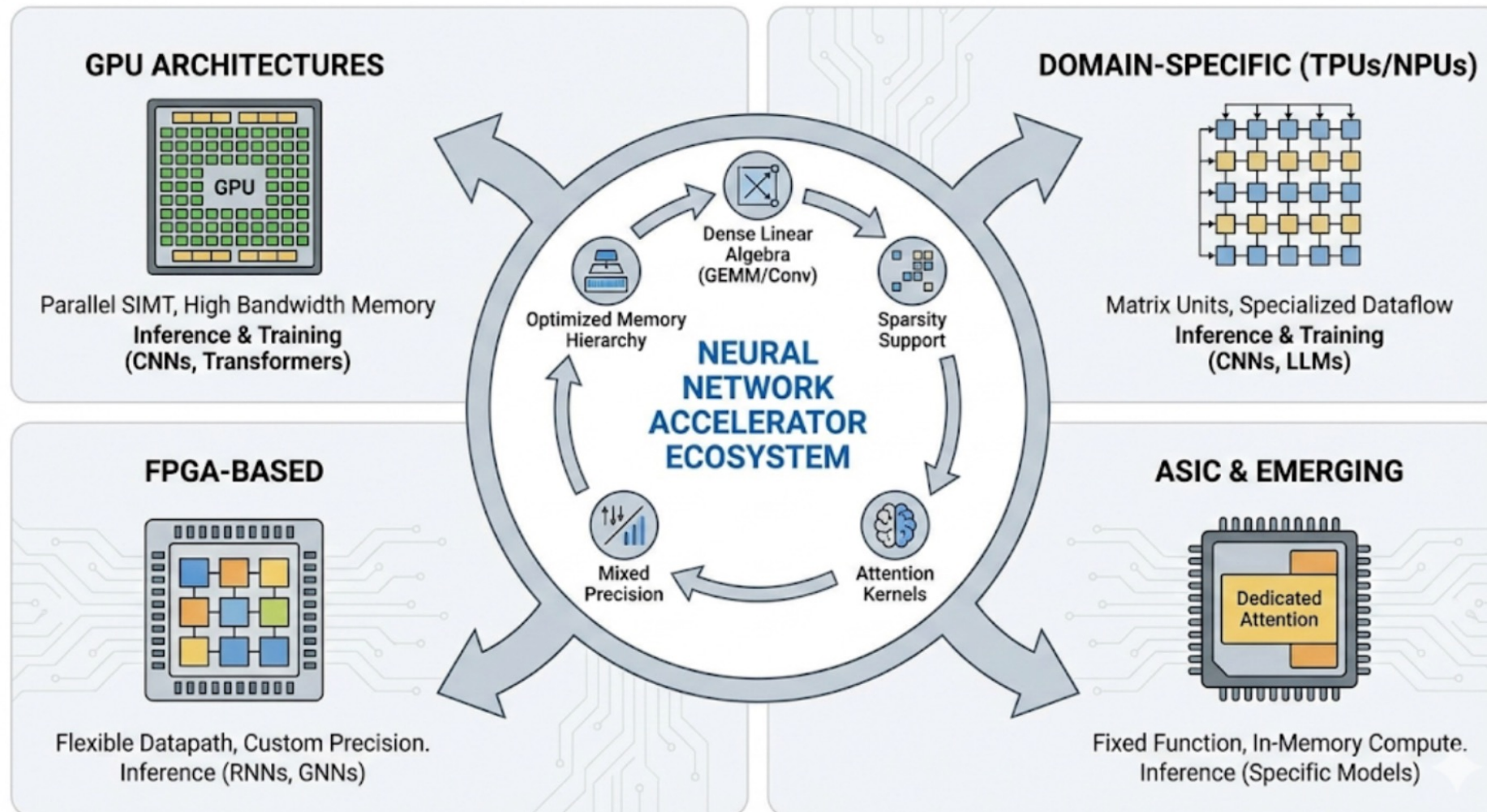
Cuts both dynamic and leakage energy with no accuracy cost.

4. Format-aware drivers (hardware)

Stream x in CSR (Compressed Sparse Row) and CSC (Compressed Sparse Column) order.

Drive only the rows that match active non-zero patterns in each tile.

Neural network acceleration

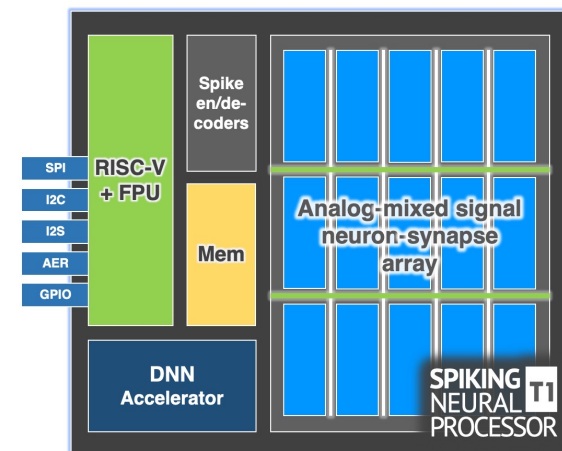
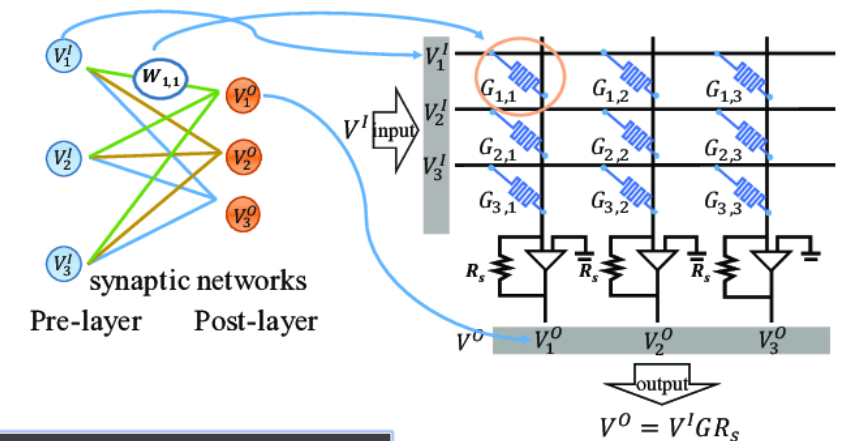
**QUIZ**

<https://arxiv.org/abs/2512.23914>

Key building block: the crossbar

QUIZ

- Crossbar arrays provide the critical architecture for efficiently implementing synaptic connections between neurons.
- Why?
 - Efficient matrix-vector multiplication
 - In-memory computing
 - Implementation of synaptic weights
 - Parallel processing
 - Integration with emerging technologies



Generic diagram?

It's all about AER (Address Event Representation)

- Address Event Representation (AER) is a spike event message passing protocol for Network-on-Chip (NoC).
- Almost all hardware and software simulation environments use a variant of AER
- The simplest and most efficient implementation of AER sends a firing neuron's unique identification number to all the nodes containing any of that neuron's targets.
- Each spike is typically encoded as a packet containing source/destination information
- Only active neurons that produce spikes generate messages.
- The timing information is implicit in when the message is sent.
- Efficient, event-driven communication system.
- AER allows for asynchronous, sparse communication between neuromorphic cores
- A NoC routes these spike event messages between different neural processing elements.

BOAHEN: POINT-TO-POINT CONNECTIVITY BETWEEN NEUROMORPHIC CHIPS

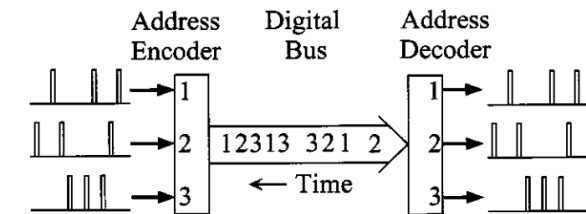


Fig. 1. The AER pulses from spiking neurons are transmitted serially by broadcasting addresses on a digital bus. Multiplexing is transparent if the encoding, transmission, and decoding processes cycle in less than Δ/n s, where Δ is the desired spike-timing precision and n is the maximum number of neurons that are active during this time (adapted from [4]).

Point-to-Point Connectivity Between Neuromorphic Chips Using Address Events

Kwabena A. Boahen

Network-on-Chip (NoC)

Packet-switched interconnect for many-core and neuromorphic chips

QUIZ

What it is

- ▶ On-chip packet-switched network replacing buses & point-to-point wires
- ▶ Cores connect via routers; data travels as flits/packets across links

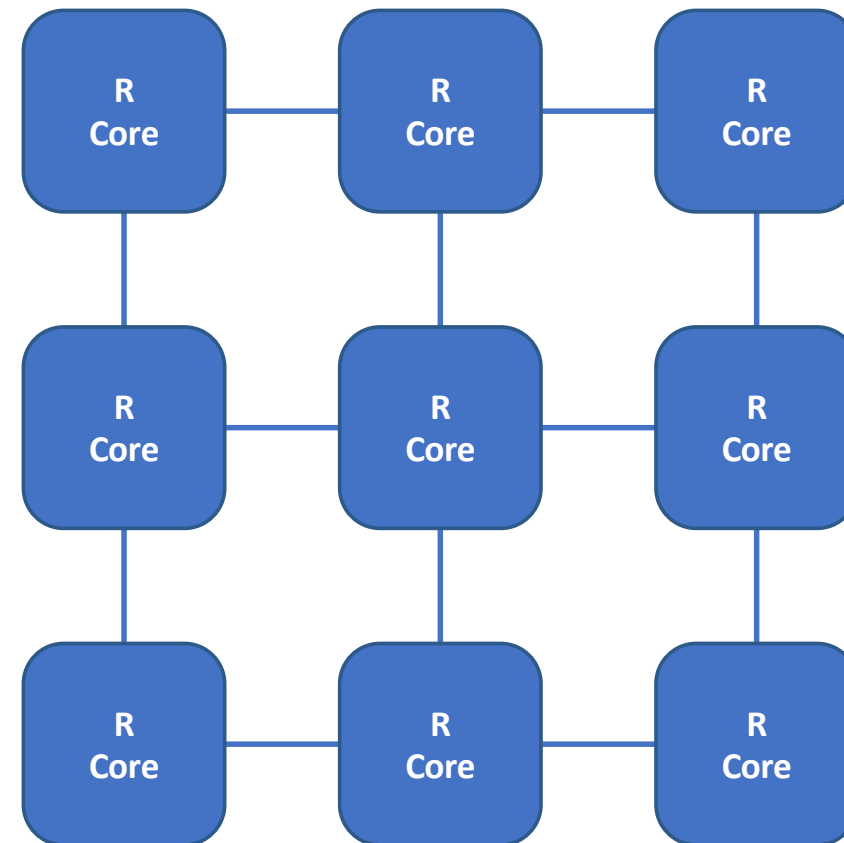
Components

- ▶ **Router:** arbitration, buffering, switching
- ▶ **Link:** physical wires between routers
- ▶ **NI:** Network Interface, packs/unpacks data

Why use it?

- ▶ Scalability: bandwidth grows with router count
- ▶ Parallelism: multiple concurrent transactions
- ▶ Modularity: drop-in IP blocks exist

2D Mesh Topology (3×3 example)



Each node = router + processing core. Lines = bidirectional links

Other topologies: torus, ring, tree, butterfly, hierarchical

Example: AER over NoC

Routing a spike from neuron (2,0) on Core A to neuron (5,3) on Core E

What is AER?

Address Event Representation: when a neuron spikes, the system sends only the spiking neuron's **address** as an event packet over the NoC.

AER packet format

| DEST_CORE | NEURON_ID | TIMESTAMP |

Spike event from A→E:

DEST = E (2,1)

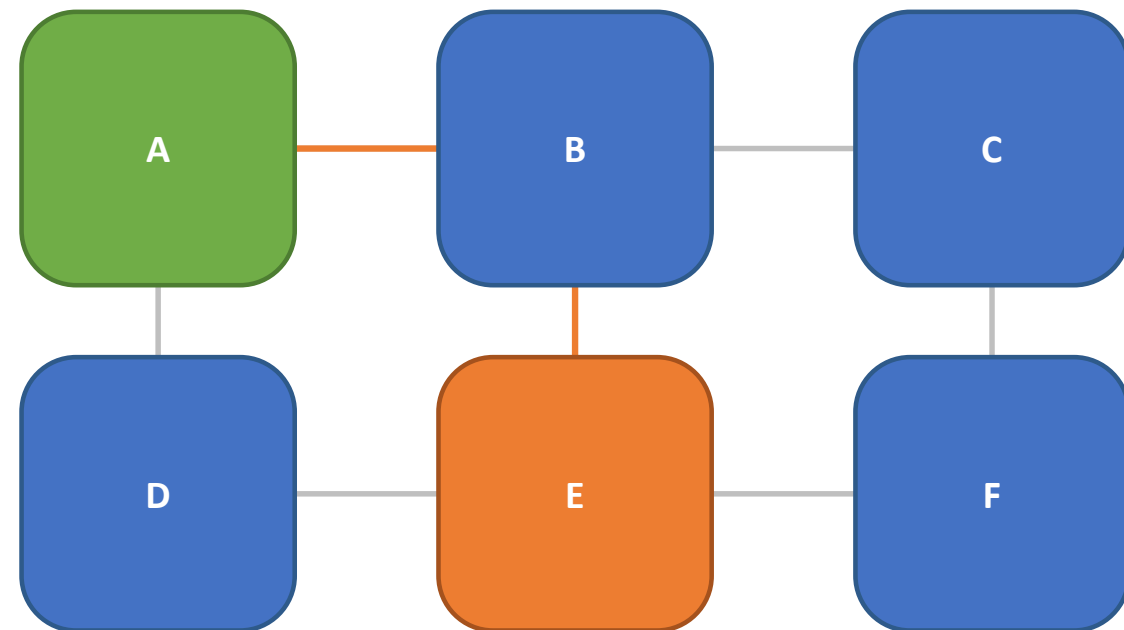
NID = neuron #(5,3)

T = 1.247 ms

Routed via XY: east → east → south

QUIZ

Spike route across the mesh



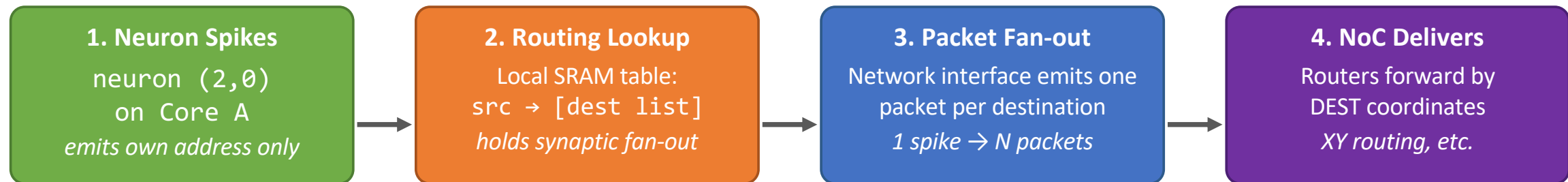
■ Source core (A) ■ Destination core (E) → spike packet path

Sparse, asynchronous: routers fire only when a spike occurs. Low average power, high peak bandwidth (used in IBM TrueNorth, Intel Loihi, SpiNNaker)



How Does the Source Know the Destination?

It doesn't — a routing table at the source core looks up the fan-out



Example routing table on Core A

SOURCE	DESTINATION FAN-OUT
(2,0)	E:(5,3), F:(1,7), B:(0,2)
(2,1)	D:(4,1), E:(2,8)
(2,2)	C:(6,0), F:(3,4), F:(3,5)
...	(stores synaptic connectivity)

Key insights

- ▶ **Neurons are "dumb"** — they only know their own ID
- ▶ **Connectivity = table** — synapses live in SRAM, not in wires
- ▶ **Programmable topology** — rewrite table = rewire network
- ▶ **Fan-out at source** — enables on-line learning (STDP)

Real chips: SpiNNaker (multicast routers), Loihi (axon table), TrueNorth (per-neuron dest list)

CSR: Compressed Sparse Row

Example matrix A

	j=0	j=1	j=2
i=0	5	0	0
i=1	0	0	8
i=2	0	3	0

values *the nonzeros*

5	8	3
---	---	---

col_idx *column of each NZ*

0	2	1
---	---	---

row_ptr *where row i starts*

0	1	2	3
---	---	---	---

QUIZ

- col_idx stores coordinates directly. row_ptr does not.
- col_idx[k] = the column of the k-th nonzero. Direct coordinate, one entry per NZ.
- row_ptr does not store "the row of each NZ." It would be wasteful to repeat the row index for every NZ in the same row. Instead, it stores where each row's run of nonzeros starts in the values array. Length N+1, not nnz.

3 is sentinel. It equals the total nonzero count.

How to read row i

Slice `row_ptr[i]..row_ptr[i+1]` gives the nonzeros in row i.

Row 1: `row_ptr[1..2] = [1,2]`

→ one NZ at index 1: `values[1]=8` at column `col_idx[1]=2`.

Memory: $\sim 2 \cdot \text{nnz} + N$ values instead of N^2 . Walk rows in order → ideal for $y = A \cdot x$.

Compared to Coordinate format (COO)

COO stores (`row`, `col`, `val`) for every NZ — $3 \cdot \text{nnz}$ entries.

CSR keeps `col_idx` explicitly but compresses the row coordinate into `row_ptr` (N+1 entries, not nnz).

= COO with the row column run-length-encoded.

CSR Example: 4×4 with 2 NZ per row

QUIZ

Matrix A					values							
4×4, 2 NZ per row					the 8 nonzeros, row by row							
	j=0	j=1	j=2	j=3	k=0	k=1	k=2	k=3	k=4	k=5	k=6	k=7
i=0	5	0	0	3	5	3	8	2	6	9	1	7
i=1	0	8	2	0	0	3	1	2	0	3	1	3
i=2	6	0	0	9								
i=3	0	1	0	7								

col_idx								row_ptr				
column of each NZ								start of each row in values (N+1 = 5)				
0	3	1	2	0	3	1	3	0	2	4	6	8

sentinel = number nz

- **row_ptr[i]** answers "what value of k does row i start at?"
- It's a bookmark into the k-indexed arrays.

How to read row i

Slice **row_ptr[i]..row_ptr[i+1]** gives that row's nonzeros.

Row 2: **row_ptr[2..3] = [4,6]** → NZs at k = 4, 5.

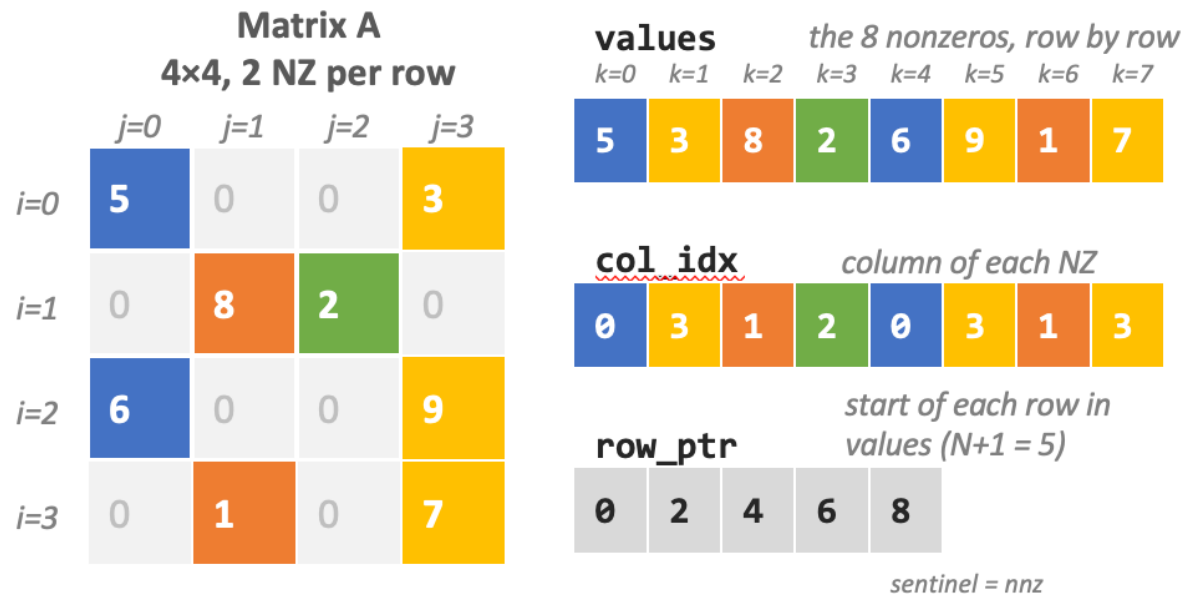
values[4]=6 at **col_idx[4]=0**; **values[5]=9** at **col_idx[5]=3**.

Storage: $8 + 8 + 5 = 21$ ints, vs 16 for dense. CSR wins as N grows.

Read-out: A in (i, j, value) form

(0,0,5)	(0,3,3)	row 0 starts at k=0
(1,1,8)	(1,2,2)	row 1 starts at k=2
(2,0,6)	(2,3,9)	row 2 starts at k=4
(3,1,1)	(3,3,7)	row 3 starts at k=6

Reconstructing A from CSR



- **row_ptr[i]** answers "what value of k does row i start at?"
- It's a bookmark into the k-indexed arrays.

QUIZ

Walk by row_ptr

```
for i in 0..N-1:
    for k in row_ptr[i]..row_ptr[i+1]-1:
        A[i][col_idx[k]] = values[k]
```

Trace on the 4x4 example:

i=0: row_ptr[0..1]=[0,2] → k=0,1 → A[0][0]=5, A[0][3]=3
 i=1: row_ptr[1..2]=[2,4] → k=2,3 → A[1][1]=8, A[1][2]=2
 i=2: row_ptr[2..3]=[4,6] → k=4,5 → A[2][0]=6, A[2][3]=9
 i=3: row_ptr[3..4]=[6,8] → k=6,7 → A[3][1]=1, A[3][3]=7

All 8 NZs placed; A fully reconstructed.